Software Engineering Department

Braude College of Engineering

Capstone Project – Phase B (61999)

Title: Atmospheric Simulation

Subtitle: A 3D Particle-Based Planetary Atmosphere Model

Code: 26-1-R-13

By: Yasser Sadi & Diana Hujerat

Advisor: Dr. Zakharia Frenkel

Link to GitHub:

https://github.com/Yasser-Saadi-7/Atmospheric-Particle-Motion-Simulation-3D-Environmental-Modeling.git

Table of Content

Document Control

Item	Details
Document title	AtmosphericSimulation - Capstone Project Phase B
Version	1.0 - Submission Version
Authors	Yasser Sadi and Diana Hujerat
Advisor	Dr. Zakharia Frenkel
Team code	26-1-R-13
Primary repository	Atmospheric-Particle-Motion-Simulation-3D-Environmental-Modeling
Main implementation	C++ / CMake
Visualization stack	Python, Pandas, NumPy, Matplotlib, Plotly, ImageIO
Primary execution environment	Windows with WSL Ubuntu; Linux-compatible

Abstract

AtmosphericSimulation is a Lagrangian, molecular-dynamics-inspired model of a planetary atmosphere; the atmosphere is represented by 10,000 macroscopic air parcels moving within a spherical shell, with parcel motion governed by radial gravity, short-range repulsive interactions, radial boundary reflection, stochastic thermal relaxation, and a one-time rotational velocity imprint, and the system also includes per-particle specific humidity, near-surface evaporation, condensation, latent-heating feedback, spherical diagnostics, a meridional circulation accumulator, and a stream function-like diagnostic; developed as an educational and research-oriented simulation framework rather than a quantitative weather-prediction model, its principal engineering contribution is the integration of a C++ simulation engine, structured CSV outputs, an offline Python visualization pipeline, interactive 3D views, report-ready figures, animation, and a local web dashboard, with a clean-run pipeline that automates compilation, removal of stale output, simulation execution, output verification, and dashboard generation. The completed system demonstrates stable long-duration execution, spherical particle transport, equator-to-pole thermal forcing, rotation-aware diagnostics, and a simplified moisture cycle; the circulation output shows organized meridional structure, but the available evidence is not sufficient to claim a fully developed Earth-like Hadley-Ferrel-Polar three-cell circulation, and moisture results also reveal a configuration limitation: evaporation and condensation are active from initialization rather than being introduced only after dry circulation has stabilized.

Submission Deliverables

Deliverable	Location / Evidence
Operational software	C++ source, CMake build, run_full_pipeline.sh
Project book	Main body of this document
User guide	Appendix A
Maintenance guide	Appendix B
Visualization website	visualization_output/index.html
Scientific outputs	CSV, PNG, HTML, GIF, VTK
Poster and video	Submitted separately in the Git repository

List of Abbreviations

Abbreviation	Meaning
CSV	Comma-Separated Values
GCM	General Circulation Model
MD	Molecular Dynamics
WSL	Windows Subsystem for Linux
VTK	Visualization Toolkit
GUI	Graphical User Interface
API	Application Programming Interface
Ψ	Meridional overturning streamfunction diagnostic
q_p	Parcel specific humidity
v_r / v_θ / v_φ	Radial, meridional, and zonal velocity components

1. Problem Statement and Background

1.1 Motivation

Atmospheric circulation is a complex three-dimensional process driven by gravity, thermal gradients, rotation, and moisture exchange. Conventional atmospheric models tackle this by solving continuous fluid equations on large numerical grids. This project takes a different, educational approach: representing the atmosphere as a collection of interacting macroscopic parcels whose collective motion gives rise to observable transport patterns.

The central engineering challenge was building a complete simulation framework that stays physically interpretable, remains numerically stable over long runs, stays computationally feasible at 10,000 parcels, and is usable by other students or reviewers. The system therefore had to bring together physics, algorithms, diagnostics, visualization, automation, and documentation, rather than focusing on just a single numerical kernel.

1.2 Why a Particle-Based Model?

A Lagrangian model follows individual air parcels rather than fixed grid points. This makes parcel trajectories, thermal histories, spherical velocity decomposition, and humidity transport conceptually direct to track. Each parcel stores its own position, velocity, acceleration, temperature, mass, and specific humidity. Pressure-like behavior is approximated through local repulsion rather than a continuous pressure field.

1.3 Background from Phase A

Phase A surveyed Eulerian and Lagrangian atmospheric modeling, parcel transport, global circulation, thermal forcing, rotating-frame effects, moisture, and numerical integration. Phase B then translated that conceptual design into a working C++ system with saved outputs and post-processing. The final architecture stays true to the Phase A principle that large-scale motion should emerge from parcel dynamics rather than being prescribed as a fixed wind field.

2. Project Objectives, Users, and Requirements

2.1 Objectives

- Represent a planetary atmosphere as particles moving within a three-dimensional spherical shell.
- Achieve stable, hydrostatic-like stratification using gravity, local repulsion, and a controlled spin-up phase.
- Build an equator-to-pole and altitude-dependent thermal forcing field.
- Introduce planetary rotation exactly once, through an inertial-frame formulation, without relying on an explicit Coriolis pseudo-force.
- Compute spherical velocity diagnostics along with a latitude-altitude streamfunction-like field.
- Add a simplified moisture cycle that handles humidity transport, evaporation, condensation, latent heating, and water-balance diagnostics.
- Provide visualization outputs that are interactive, report-ready, and easy to transfer or share.
- Make it possible to build and run the entire project through a single reproducible pipeline.

2.2 Intended Users

User Group	Primary Need
Students	Understand particle-based atmospheric dynamics and inspect saved results.
Researchers / instructors	Explore qualitative parameter effects and evaluate diagnostics.
Project reviewers	Build, run, inspect, and demonstrate the complete system.
Future maintainers	Modify physics, diagnostics, and visualizations using documented interfaces.

2.3 Key Non-Functional Requirements

- Reproducibility: a clean run shouldn't mix old and new CSV or website files.
- Maintainability: the simulation, analysis, output, and visualization components stay separated from one another.
- Usability: a reviewer should be able to run the project using documented commands and browse the results through a dashboard.
- Numerical robustness: long production runs need to complete without non-finite values or uncontrolled instability.
- Traceability: important claims are tied back to saved diagnostic files and figures.

3. Solution Overview and System Architecture

The completed solution is split into four layers: a C++ simulation engine, a saved-output layer, a Python visualization pipeline, and a result-delivery layer. This separation is deliberate — visualization is generated from saved data after the production run, so graphical work never slows down or destabilizes the numerical loop.

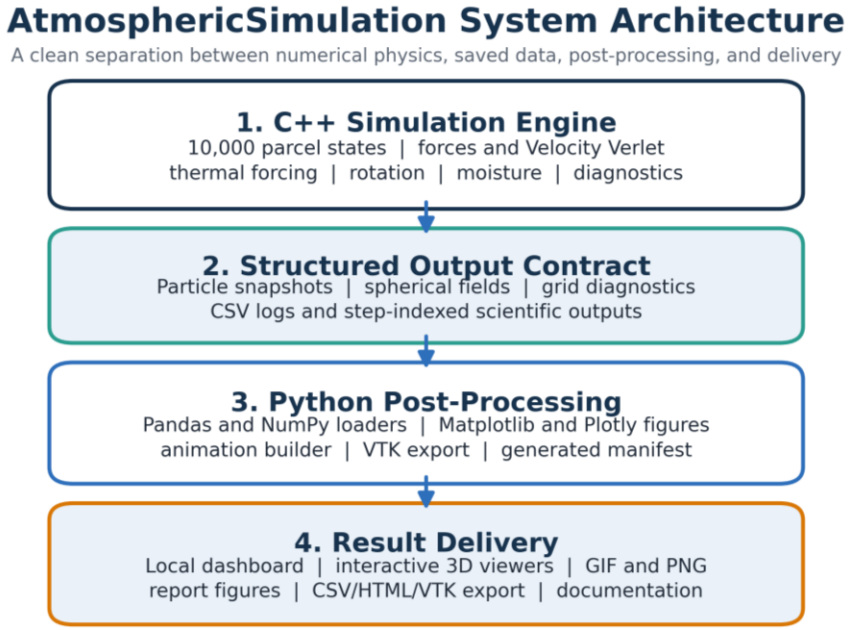

Figure 1. High-level system architecture and separation of simulation, output, analysis, and delivery layers.

3.1 C++ Simulation Engine

The engine stores parcel states, builds fine and coarse grids, computes forces, advances the system with Velocity Verlet, applies boundary conditions and thermal interactions, activates rotation, updates moisture, and writes out diagnostics. The implementation itself is organized into model, simulation, analysis, I/O, and utility modules.

3.2 Saved Output Contract

CSV files serve as the stable interface between the physics engine and the visualization side. Particle snapshots, spherical components, coarse-grid fields, temperature-zone time series, circulation accumulations, streamfunction fields, evaporation, condensation, and water-balance files can all be inspected independently of the C++ executable.

3.3 Visualization and Dashboard

Python scripts load the CSV outputs, pick out the latest valid step numerically, generate plots and interactive HTML files, build animations, create report figures, export VTK data, and update a generated manifest that the dashboard relies on. The dashboard itself groups overview information, primary results, diagnostics, figure galleries, and export links.

4. Engineering Development Process

Development followed a staged process of implementation and verification. Each stage introduced just a limited set of mechanisms and diagnostics, so that errors could be isolated before any additional complexity was added.

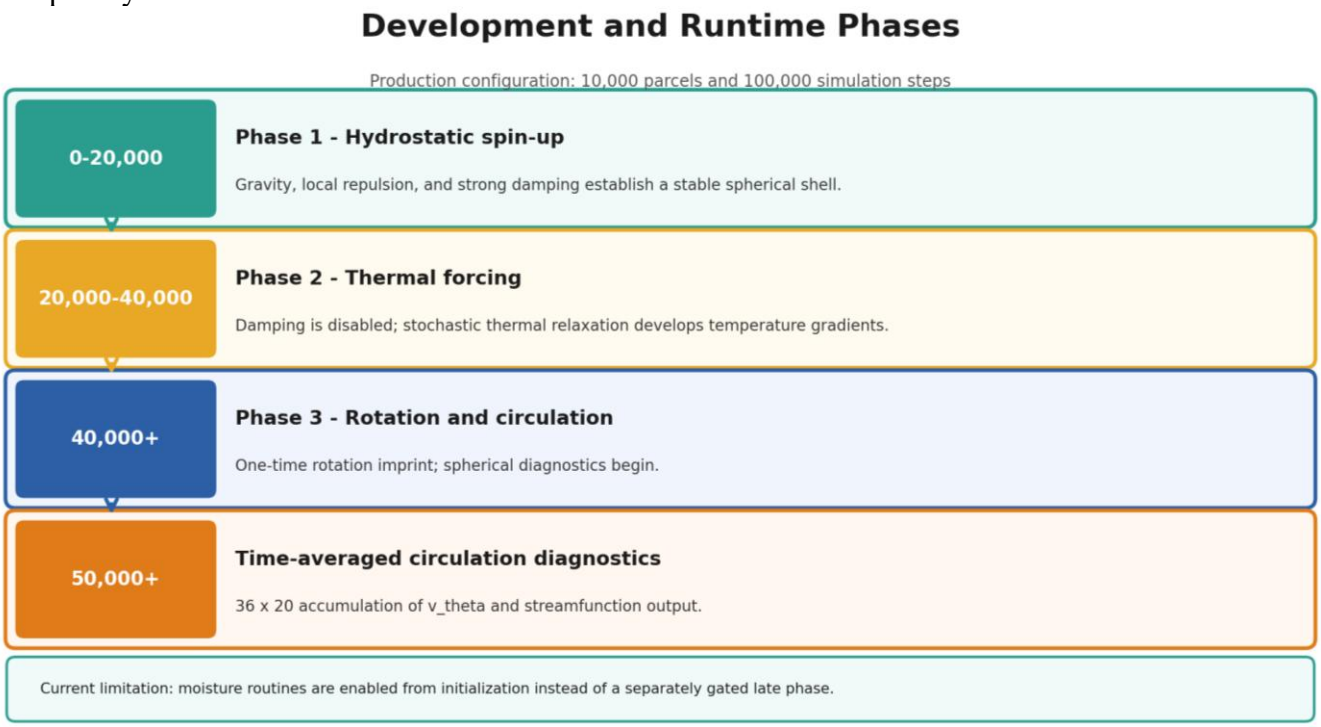

Figure 2. Runtime phases and the point at which new mechanisms and diagnostics are introduced.

4.1 Stage-Based Development

Stage	Engineering Goal	Primary Evidence
Stage 1	Stable 3D spherical shell and hydrostatic-like spin-up	Particle bounds, radial density, energy diagnostics
Stage 2	Thermal gradient and convective response	Temperature-zone and altitude profiles, radial velocity
Stage 3	Rotation-aware spherical diagnostics and circulation analysis	v_θ, v_φ, circulation accumulator, streamfunction
Stage 4	Simplified water-vapor cycle	q_p, evaporation, condensation, latent heating, water balance
Visualization	Reusable scientific result delivery	Dashboard, interactive HTML, PNG/GIF, report figures, VTK

4.2 Tools and Technologies

Area	Tools
Core implementation	C++17, CMake, standard library
Numerical and spatial algorithms	Velocity Verlet, 3D spatial hashing, coarse aggregation
Analysis and visualization	Python, Pandas, NumPy, Matplotlib, Plotly, ImageIO
Development environment	Windows, WSL Ubuntu, VS Code / Cursor
Version control	Git and GitHub
Scientific export	CSV and VTK / ParaView

5. Physical Model and Core Algorithms

5.1 Atmospheric Domain and Parcel State

The model domain is the shell $R \leq |r| \leq R + H$, with $R = 50$ and $H = 20$ model units. A production run uses 10,000 parcels. The shell is intentionally thicker than Earth's real atmosphere relative to planetary radius; this increases the visual and diagnostic resolution of vertical motion but limits direct physical comparison.

Component	Implemented Model	Engineering Role
Repulsion	$F_{ij} = k(\sigma^2 - r_{ij}^2)\hat{r_{ij}}\ for\ r_{ij} < \sigma, with\ force\ cap$	Prevents overlap and creates pressure-like behavior
Gravity	$F_{grav} = -mg\hat{r}$	Produces inward acceleration and density stratification
Damping	$F\_damp = -\gamma(v - \Omega \times r), Phase\ 1\ only$	Suppresses initialization transients
Boundary handling	Radial projection and reflection at R and R+H	Keeps all parcels inside the shell
Thermal forcing	Stochastic relaxation toward T_{target} (θ, h)	Generates horizontal and vertical thermal gradients
Rotation	One-time addition of Ω×r at Phase 3 start	Seeds solid-body rotation in the inertial frame

5.2 Numerical Integration

Velocity Verlet advances the velocity by a half step, updates the position, applies radial boundary handling, rebuilds the fine grid, recomputes the forces, and then completes the second velocity half step. The method is second order and well suited to particle dynamics. The time step dt = 0.01 is kept substantially smaller than the approximate stability limit associated with the repulsive stiffness.

5.3 Two-Level Grid

The fine grid uses cells of size σ and searches the current cell plus 26 neighbors. This avoids an $O(N^2)$ all-pairs force calculation. The coarse grid aggregates mean velocity, kinetic temperature, target temperature, and mean humidity for thermal interactions and diagnostics.

5.4 Circulation and Streamfunction Diagnostic

After step 50,000, parcel samples are accumulated into a 36 × 20 latitude-altitude grid. The proxy meridional mass flux is computed as particle count multiplied by mean meridional velocity. A vertical cumulative integral generates Ψ. This is a kinematic diagnostic using particle count as a density proxy; it is not a fully mass-conserving atmospheric Stokes streamfunction.

5.5 Moisture Model

Specific humidity q_p is carried with each parcel. Near-surface parcels receive moisture when subsaturated, supersaturated parcels condense, and condensation adds model-scaled latent heat to parcel temperature. Thermal collisions relax parcel humidity toward the coarse-cell mean. The saturation computation maps model temperature and altitude to Kelvin and pressure before applying a meteorological saturation-vapor-pressure relation.

6. Implementation, Execution, and Data Flow

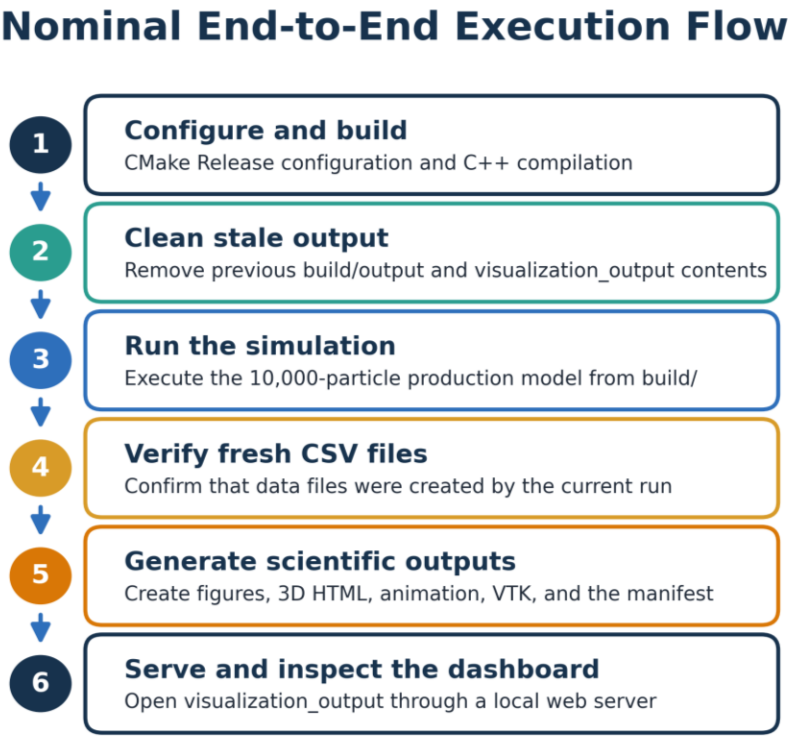

Figure 3. Nominal build, execution, verification, visualization, and dashboard flow.

6.1 Clean-Run Pipeline

The run_full_pipeline.sh script configures and builds the C++ project, removes stale files from build/output, executes the simulation from the build directory, verifies that fresh CSV files were created, removes obsolete visualization output, regenerates the dashboard, and verifies the required website assets. Safety checks prevent accidental deletion outside the project directory.

6.2 Main Output Files

Output	Purpose
particles_step_*.csv	Cartesian particle state and basic thermodynamic variables
particle_spherical_step_*.csv	Latitude, longitude, altitude, and spherical velocity components
coarse_grid_step_*.csv	Macroscopic fields used by thermal and moisture analysis
temperature_zones.csv	Equatorial, mid-latitude, and polar temperature evolution
circulation_accum_step_*.csv	Time-averaged meridional statistics
streamfunction_step_*.csv	Latitude-altitude Ψ field
streamfunction_summary.csv	Circulation-strength summary over time
evaporation_log.csv / condensation_log.csv	Moisture sources, sinks, events, and latent heating
moisture_balance.csv	Actual and expected total humidity

6.3 User-Facing Outputs

- Interactive 3D particle viewers, colored by temperature, radial velocity, meridional velocity, or humidity.
- Interactive particle animation with fixed geometry and a choice of variables to display.
- Latitude-altitude heatmaps and streamfunction contours.
- Time-series diagnostics for circulation and moisture.
- Report-ready PNG figures and downloadable data files.
- VTK output meant for ParaView-based scientific rendering.

7. Testing and Evaluation

7.1 Test Strategy

Testing combined code-level checks, runtime assertions, CSV inspection, external analysis, and visual review. The simulation and visualization layers were evaluated separately so that a graphical defect could not be confused with a physics defect.

Test Area	Method	Acceptance Evidence
Build and execution	Release build and 100,000-step run	Process completes and produces current-run CSV files
Shell geometry	Minimum/maximum radius and out-of-shell counts	Particles remain within the defined spherical boundaries
Integrator	Energy test with damping and thermal collisions disabled	Energy drift remains within the configured tolerance
Thermal forcing	Zone and altitude profiles	Equatorial target is warmer than polar target; temperature decreases with altitude
Rotation	Source inspection and runtime transition	$\Omega \times r$ is added exactly once; no Coriolis pseudo-force is present
Circulation	Finite v_θ, mass-flux proxy, and Ψ outputs	No NaN/inf; reproducible latitude-altitude fields
Moisture	Evaporation, condensation, q_p bounds, and water-balance logs	Finite, bounded values and interpretable accounting status
Visualization	Regeneration from saved outputs and browser checks	Dashboard assets exist and interactive pages open

7.2 Known Verification Gaps

The VTK exporter previously produced an incomplete file, so it still needs one final independent ParaView opening test before submission. On top of that, the circulation diagnostic has to be assessed over several late-time intervals before canonical circulation-cell labels can be used. Both items are included in the final submission checklist.

8. Results and Interpretation

8.1 Particle Shell and Visualization

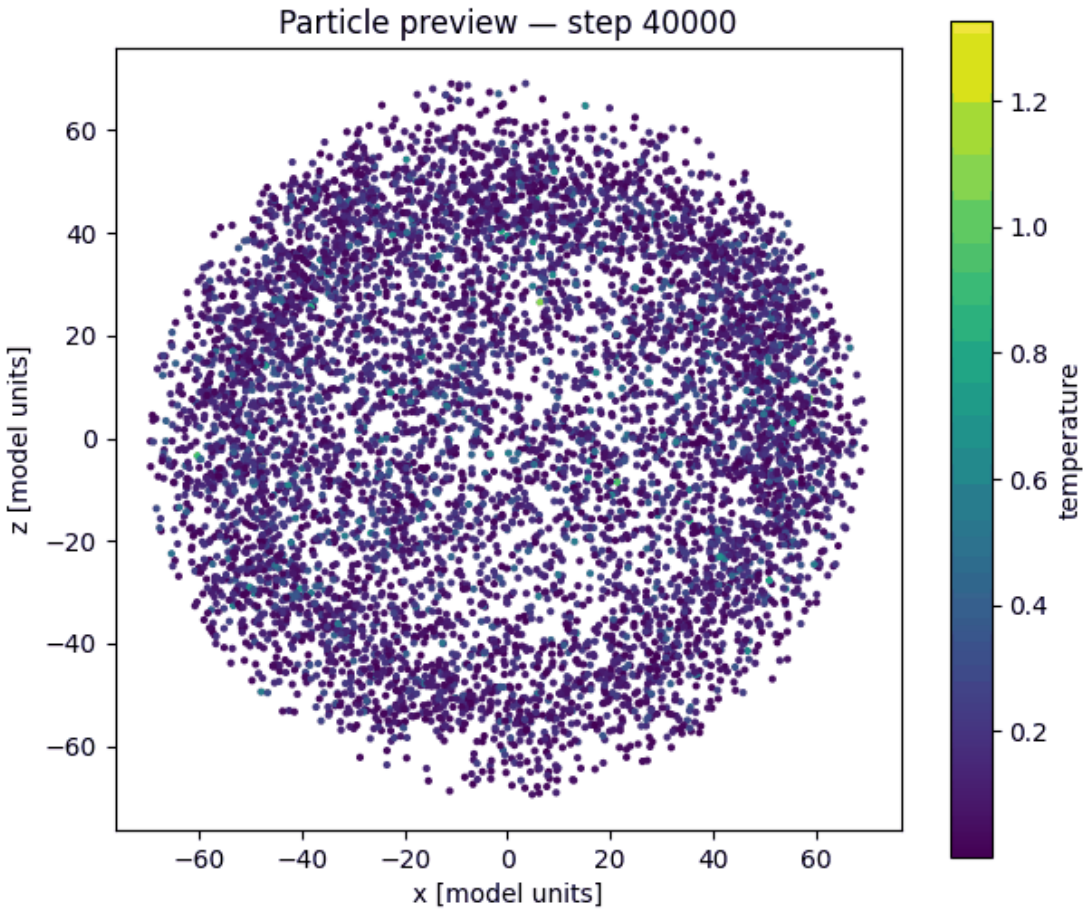

Figure 4. Saved particle-animation preview showing the populated atmospheric shell. The website provides an interactive 3D version.

The production configuration runs with 10,000 parcels out to step 100,000. Interactive visualizations may downsample the displayed particles for the sake of browser performance, but this doesn't change the number of parcels actually used by the simulation engine.

8.2 Meridional Circulation Diagnostics

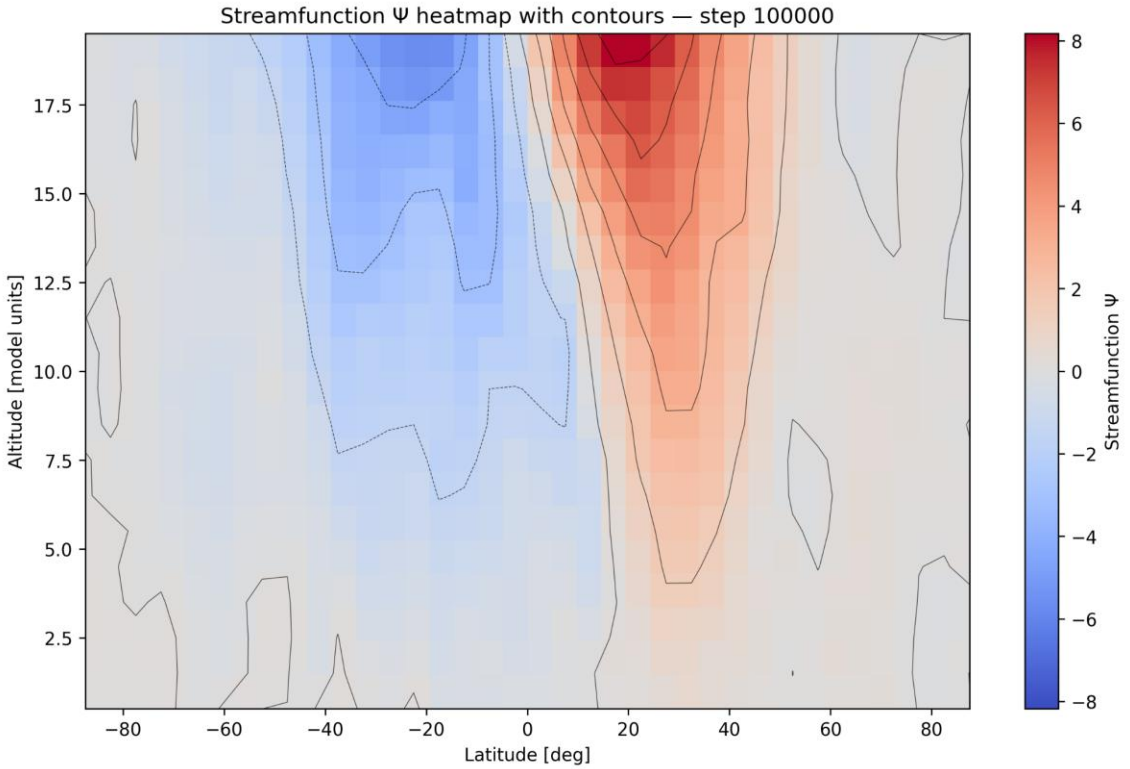

Figure 5. Streamfunction-like diagnostic Ψ in latitude-altitude coordinates with contours.

The streamfunction output is finite and non-zero and displays organized positive and negative regions. This supports the presence of structured meridional transport.

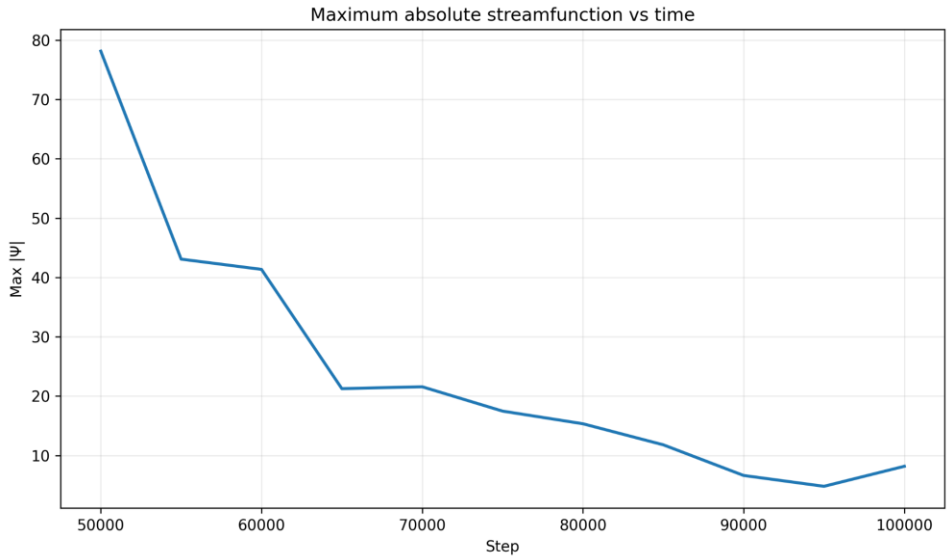

Figure 6. Evolution of the maximum absolute streamfunction amplitude.

The amplitude time series provides a compact measure of circulation strength and helps distinguish persistent development from a single visually strong snapshot. It should be interpreted together with the spatial Ψ map and meridional-velocity field.

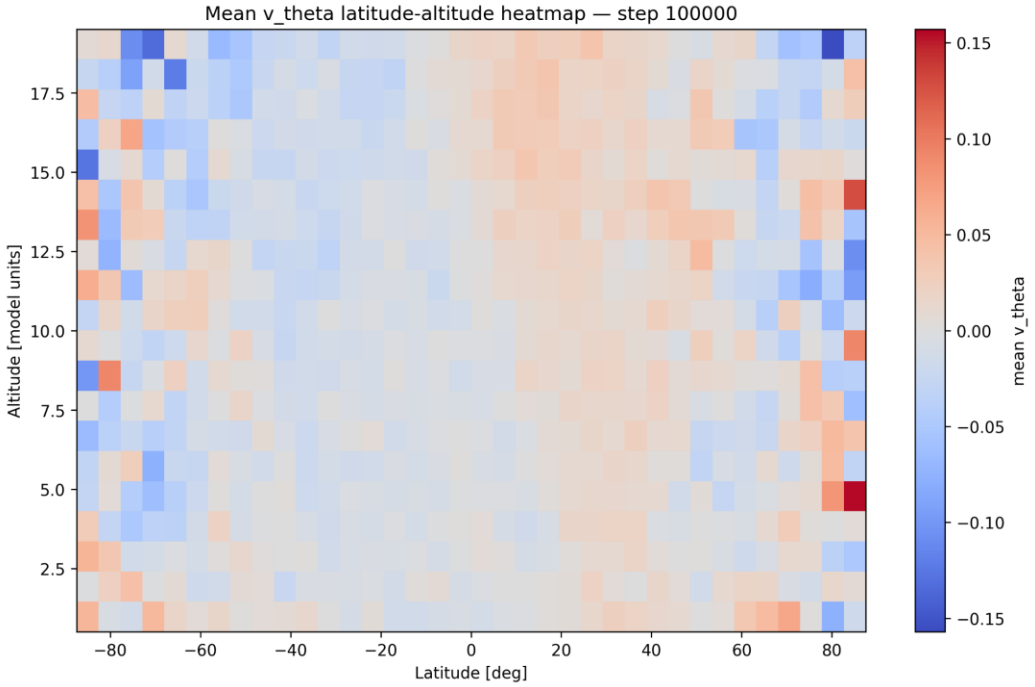

Figure 7. Longitude-averaged meridional velocity v_θ across latitude and altitude.

Alternating northward and southward regions are visible in the meridional-velocity diagnostic. The field is useful for checking consistency with Ψ and for identifying possible overturning branches.

8.3 Moisture Results

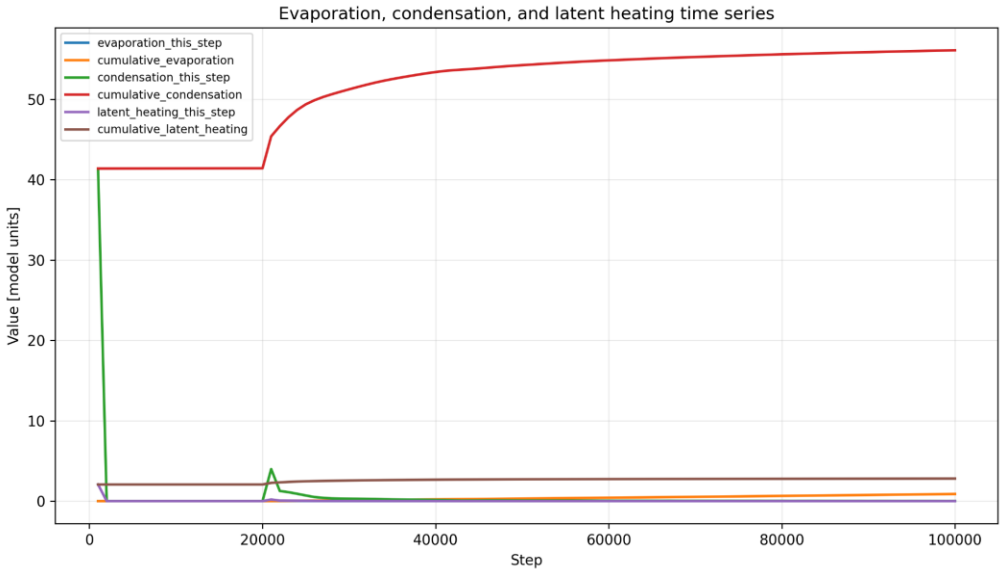

Figure 8. Evaporation and condensation diagnostics across the 100,000-step run.

At step 100,000, cumulative evaporation is approximately 0.887 model humidity units, whereas cumulative condensation is approximately 56.108. The final logged interval contains about 0.0127 evaporation and 0.0220 condensation. Cumulative model-scaled latent heating reaches approximately 2.805.

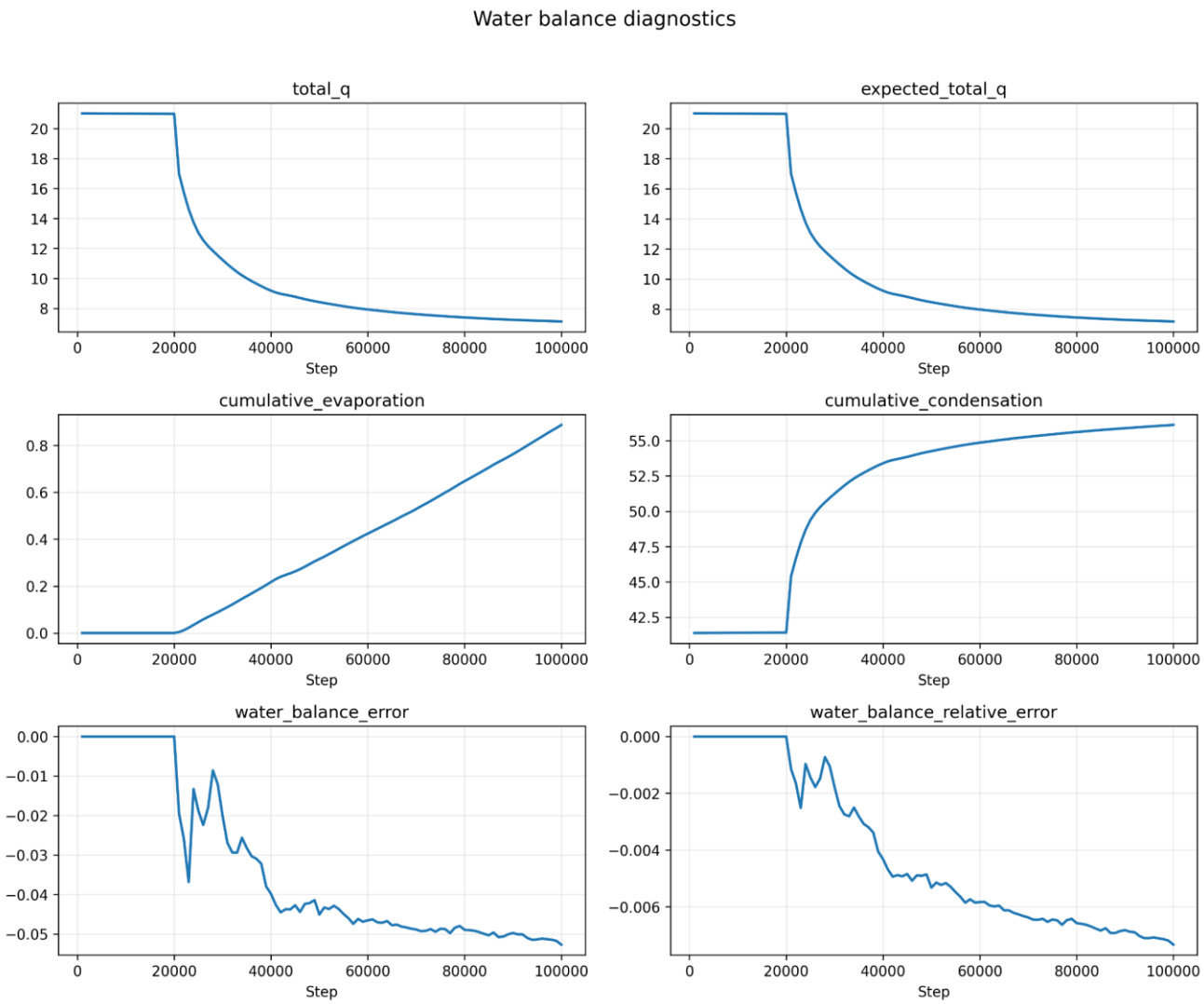

Figure 9. Water-budget and moisture diagnostic panels.

9. Engineering Challenges and Decisions

Challenge	Decision / Solution	Reasoning
Particle containment	Radial projection and radial-velocity reflection at both shell boundaries	Prevents leakage while preserving tangential motion
Computational cost	3D fine-grid neighbor search instead of all-pairs interaction	Reduces force computation from $O(N^2)$ to local occupancy scaling
Initialization instability	Strong Phase 1 damping, disabled permanently afterward	Allows the shell to settle without suppressing later dynamics
Rotation modeling	One-time $\Omega \times r$ imprint in the inertial frame	Avoids double-counting and removes the need for a pseudo-force
Noisy circulation	Long-time latitude-altitude accumulation before Ψ calculation	Improves diagnostic stability
Stale data contamination	Delete old simulation and visualization output before a clean run	Guarantees traceability to the latest execution
Browser performance	Downsample display data while keeping full simulation data	Preserves scientific computation and maintains interactivity
Heatmap visibility	Data-driven percentiles and symmetric zero-centered limits where appropriate	Prevents valid structure from appearing blank
AI-generated code risk	Git checkpoint, diff review, focused prompts, rebuild and output verification	Keeps human control over the source and results

10. Project Metrics and Achievement

Metric	Target	Observed Status	Assessment
Complete production run	100,000 steps	Completed	Achieved
Parcel population	10,000 simulation parcels	Implemented	Achieved
3D spherical geometry	$R \leq r \leq R+H$	Implemented with radial boundaries	Achieved; final CSV check remains required per run
Reproducible pipeline	One command from clean output to dashboard	run_full_pipeline.sh	Achieved
Thermal diagnostics	Latitude and altitude temperature outputs	Generated	Achieved
Rotation handling	Apply $\Omega \times r$ once; no pseudo-force	Confirmed by code design	Achieved
Circulation diagnostic	Finite time-averaged v_θ and Ψ	Generated and visualized	Achieved as a diagnostic
Earth-like three-cell circulation	Stable canonical cell structure	Not conclusively demonstrated	Partially achieved / not claimed
Moisture processes	Evaporation, condensation, latent heat, water accounting	Generated	Achieved with configuration and balance limitations
Transferable UI	Dashboard, interactive pages, export package	Implemented	Achieved
ParaView compatibility	Valid VTK opens without error	Requires final test	Open item

11. Lessons Learned and Future Work

11.1 Lessons Learned

- Define the output schemas and success metrics before implementing the late-stage diagnostics.
- Keep simulation output separate from visualization artifacts, and regenerate both through a clean pipeline.
- Treat every figure as a view of the data, not as evidence on its own.

- Use late-time averaging for circulation, and verify it across multiple intervals before interpreting it.
- Commit known-good versions before any broad AI-assisted refactoring.
- Keep the user-facing documentation separate from maintainer-level troubleshooting.

11.2 Recommended Future Work

Priority	Improvement	Expected Benefit
High	Gate moisture activation to a configurable late step	Separates dry circulation from moisture transients
High	Complete and verify valid VTK/VTU export in ParaView	Provides reliable scientific 3D export
High	Add automated end-to-end tests for required CSV columns and freshness	Prevents silent pipeline regressions
Medium	Use physical density weighting and mass-conserving streamfunction formulation	Improves circulation interpretation
Medium	Run parameter ensembles for Ω and ΔT	Tests robustness of circulation patterns
Medium	Add checkpoint save and restart verification	Supports long experiments and recovery
Long term	Calibrate model units and surface/moisture physics	Improves quantitative physical relevance
Long term	Add clouds, precipitation, geography, and topography	Extends the water-cycle and boundary model

12. Conclusions

AtmosphericSimulation shows that a complete educational atmospheric model can be built around a Lagrangian parcel representation. The project brings together a stable C++ simulation core, spatial acceleration structures, staged forcing, spherical diagnostics, moisture processes, automated saved outputs, and a professional visualization website. The strongest result of the project is the integrated engineering system itself: a reviewer can build the software, generate a fresh 100,000-step dataset, recreate the figures and interactive pages, and inspect the scientific evidence without ever touching the physics source. The project also demonstrates disciplined reporting by clearly distinguishing the mechanisms that were implemented from the outcomes that were actually validated. The simulation provides evidence of thermal gradients, rotation-aware flow, organized meridional transport, and an active moisture cycle.

References

1. Sadi, Y., & Hujerat, D. (2026). *Simulation of Atmospheric Particle Movement: A Framework for Modeling Air Parcel Dynamics* — Phase A Project Book (Course 61998). Braude College of Engineering.https://docs.google.com/document/d/1SMRnpVWj1hkPrNC73jKtmsV-yOiHTDhckx-abZZ2REk/edit?usp=sharing
2. Sadi, Y., & Hujerat, D. (2026). *AtmosphericSimulation* [Source code and project repository]. GitHub. https://github.com/Yasser-Saadi-7/Atmospheric-Particle-Motion-Simulation-3D-Environmental-Modeling.git
3. Sadi, Y., & Hujerat, D. (2026). *AtmosphericSimulation Physics Model Report* [Internal project document, prepared by source-code inspection, 14 June 2026]. Project repository, /documentation.
4. Sadi, Y., & Hujerat, D. (2026). *AtmosphericSimulation 3D Project Specification and Pre-Submission Checklist* [Internal project document]. Project repository, /documentation.
5. Braude College of Engineering. (2026). *Course 61999 Capstone Project – Phase 2 Submission Instructions*.
6. Sadi, Y., & Hujerat, D. (2026). Generated simulation CSV outputs and report figures, 100,000-step production run [Data set]. Project repository, /build/output and /visualization_output.

Appendix A - User Guide

This guide is meant for a user who wants to install, run, and inspect AtmosphericSimulation through the normal successful workflow. Detailed maintenance and failure-recovery steps can be found in Appendix B.

A.1 Supported Environment

The recommended environment is Windows with WSL Ubuntu. The C++ and Python components are portable to Linux and can be adapted to macOS or native Windows, but the supplied Bash automation is designed for WSL/Linux.

Requirement	Recommended Version / Tool
Operating environment	Windows 10/11 with WSL Ubuntu, or Linux
C++ compiler	G++ with C++17 support
Build system	CMake
Python	Python 3 with venv and pip
Python packages	Pandas, NumPy, Matplotlib, Plotly, ImageIO
Browser	Current Chrome, Edge, or Firefox

A.2 First-Time Installation

```
sudo apt update
sudo apt install -y build-essential cmake python3 python3-pip python3-venv git zip unzip

cd ~/projects/AtmosphericSimulation
python3 -m venv viz_env
source viz_env/bin/activate
pip install --upgrade pip
pip install -r visualization/requirements.txt
chmod +x run_full_pipeline.sh
```

A.3 Run the Complete Project

```
cd ~/projects/AtmosphericSimulation
source viz_env/bin/activate
bash run_full_pipeline.sh
```

A normal run performs a Release build, deletes stale simulation and website output, runs the simulation, verifies fresh CSV files, regenerates the visualization website, and checks the required dashboard assets.

A.4 Optional Commands

Command	Purpose

bash run_full_pipeline.sh --skip-animation	Faster test run without rebuilding the animation
bash run_full_pipeline.sh --open-dashboard	Request dashboard opening after generation
bash run_full_pipeline.sh --max-particles 8000	Change the visualization display limit
bash run_full_pipeline.sh --help	Display available options

A.5 Open the Results Website

```
cd ~/projects/AtmosphericSimulation/visualization_output
python3 -m http.server 8000
```

Keep this terminal open, then browse to http://localhost:8000/. From WSL, the following command opens it in the Windows browser:

```
explorer.exe http://localhost:8000/
```

A.6 Website Navigation

Section	What the User Can Do
Overview	Review run metadata and open the primary 3D result.
Results	Inspect central streamfunction, velocity, and moisture outputs.
Diagnostics	Open time-series and supporting scientific plots.
Gallery	Preview and download report-ready figures and animation.
Export	Access CSV, HTML, images, documentation, and VTK output.

A.7 Successful Run Checklist

- The CMake build completes without an error.
- The simulation reaches its configured final step.
- New CSV files exist in build/output.
- visualization_output/index.html exists.
- The website opens through localhost with full styling.
- Interactive plots and downloadable figures open correctly.
- To stop the local website, return to the server terminal and press Ctrl+C.

Appendix B - Maintenance Guide

This guide is intended for developers who will modify, test, extend, or deploy the project after the capstone submission.

B.1 Repository Structure

```
AtmosphericSimulation/
├── CMakeLists.txt
├── src/
│   ├── model/
│   ├── simulation/
│   ├── analysis/
│   ├── io/
│   └── utils/
├── visualization/
├── build/output/
├── visualization_output/
├── run_full_pipeline.sh
└── documentation/
```

B.2 Module Responsibilities

Module	Responsibility
model	Parcel state, vectors, and simulation configuration
simulation	Engine loop, integrator, forces, boundaries, grids, thermal and moisture physics
analysis	Diagnostics, circulation accumulator, and streamfunction calculation
io	CSV and scientific output writing
utils	Constants and spherical-coordinate utilities
visualization	CSV loading, plot generation, 3D HTML, animation, VTK, manifest, and dashboard build

B.3 Build and Run Manually

```
cd ~/projects/AtmosphericSimulation
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -
DCMAKE_EXPORT_COMPILE_COMMANDS=ON
cmake --build build --parallel "$(nproc)"

rm -rf build/output
mkdir -p build/output
cd build
./AtmosphericSimulation
```

B.4 Regenerate Visualization Only

```
cd ~/projects/AtmosphericSimulation
source viz_env/bin/activate
rm -rf visualization_output
mkdir -p visualization_output
python3 visualization/build_dashboard.py   --input build/output   --out visualization_output   --max-particles 5000   --verbose
```

B.5 Output Schema and Compatibility Rules

- Do not rename existing CSV columns without updating the data loader and every dependent plot.
- Step-dependent files must retain numeric step suffixes so the builder can select the latest step correctly.
- The visualization builder must read build/output, not the legacy root-level output directory.
- Generated HTML and image filenames referenced by the dashboard manifest should remain stable.
- A missing artifact should produce a specific availability message rather than an empty or misleading card.

B.6 Adding a New Diagnostic

1. Define the scientific quantity and its units or model-unit interpretation.
2. Add the C++ computation in the appropriate simulation or analysis module.
3. Export a stable CSV schema through the I/O layer.
4. Add a focused Python loader and plot function under visualization/.
5. Register the artifact in build_dashboard.py and generated_manifest.js.
6. Add a dashboard card only after a real generated artifact exists.
7. Verify finite values, data range, labels, file freshness, and browser behavior.
8. Document the diagnostic and its limitations in the project book.

B.7 Git and Safe Change Workflow

```
cd ~/projects/AtmosphericSimulation
git status
git add CMakeLists.txt src visualization run_full_pipeline.sh
git commit -m "Checkpoint before maintenance change"

# After an automated or AI-assisted edit
git status
git diff --stat
git diff
```

To discard uncommitted tracked-file changes, use git reset --hard HEAD only after confirming that no work needs to be preserved. Preview untracked-file removal with git clean -nd before using git clean -fd.

B.8 Required Regression Checks

Change Area	Minimum Regression Check
Force or integrator	Energy behavior, shell containment, finite positions and velocities
Boundary handling	No parcels outside [R, R+H]
Thermal model	Temperature-zone ordering and altitude profile
Rotation	Single activation and unchanged earlier-phase behavior
Circulation	Finite v_θ and Ψ with correct 36 × 20 grid dimensions
Moisture	q_p bounds, finite q_{sat}, evaporation/condensation monotonic cumulative values, water balance
Visualization	Clean regeneration, valid links, readable color scales, no stale placeholders
Pipeline	Fresh CSV verification and required website files

B.9 Known Limitations and Open Items

- Moisture physics is currently active from initialization; add a configurable activation step.
- The streamfunction uses particle count as a density proxy and is a qualitative diagnostic.
- Earth-like three-cell circulation is not conclusively validated.
- The full solar-flux parameter is not used because the simplified target-temperature field is active.
- Model units are not fully calibrated to Earth-scale SI units.
- Cloud droplets, precipitation, land-ocean contrast, topography, and a real equation of state are absent.
- VTK output must be checked with ParaView after any exporter change.

B.10 Troubleshooting Quick Reference

Symptom	Recommended Action
CMake not found	Install cmake and build-essential through the system package manager.
Executable not found	Inspect build output and confirm the target name in CMakeLists.txt.
No fresh CSV files	Run the executable from build/ and verify that output resolves to build/output/.
Python module missing	Activate viz_env and reinstall visualization/requirements.txt.
Website has no styling	Serve the complete visualization_output directory through python3 -m http.server.

Old cards or plots remain	Delete visualization_output and rebuild without --keep-visualization-output.
Old CSVs are mixed	Run without --keep-simulation-output or clean build/output manually.
Blank heatmap	Inspect finite values and use data-driven color limits instead of a fixed unsuitable range.
Animation flickers	Use fixed axes, stable particle IDs, and global color limits across frames.

B.11 Distribution

To transfer only the visualization website, copy the complete visualization_output directory into a distribution folder, add a START_WEBSITE.bat launcher, compress the whole folder, and test the extracted package from a new location. Never send index.html by itself.

B.12 Final Maintenance Checklist

- Create a Git checkpoint before broad changes.
- Run bash -n run_full_pipeline.sh after editing the pipeline.
- Perform a clean end-to-end run.
- Inspect the latest CSV timestamps and schemas.
- Open the dashboard through localhost.
- Open every interactive visualization and downloadable artifact.
- Verify VTK in ParaView.
- Update this project book, the user guide, and the maintenance guide.